MATLAB - Settimana 2

Scheda pratica basata sui sette codici studiati: vettori, funzioni, condizioni, accumulatori, matrici e aritmetica numerica.

1. Comandi e sintassi da ricordare

Sintassi	Uso
rand(n,1)	Vettore colonna di n numeri pseudocasuali uniformi tra 0 e 1.
zeros(n,1)	Vettore colonna di n zeri. zeros(n) crea invece una matrice n per n.
[nr,nc] = size(A)	Numero di righe e colonne di A. Non usare size come nome di variabile.
length(v)	Lunghezza di un vettore. Per una matrice restituisce la dimensione maggiore.
A(i,j)	Elemento alla riga i, colonna j. Gli indici partono da 1.
prod(v) / sum(v)	Prodotto / somma degli elementi di un vettore.
a:h:b	Sequenza con inizio a, passo h e limite b. Esempio: n:-1:1.
[]	Array vuoto. Negli esercizi viene usato come output per casi non validi.
disp(x)	Visualizza un valore o un testo.
load matriceB.mat	Carica nel workspace le variabili del file MAT.
v = eig(B)	Autovalori numerici della matrice quadrata B, in un vettore colonna.
det(B)	Determinante numerico: non usarlo da solo come test di singolarita.

2. Dal ciclo alla forma compatta

```
n = 8;
a = rand(n,1);
b = rand(n,1);
c = zeros(n,1);
for i = 1:length(a)
    c(i) = 2*a(i) - b(i);
end
% Stesso calcolo, in forma vettoriale:
c_diretto = 2*a - b;
```

Se assegni direttamente c = 2*a - b, la precedente inizializzazione c = zeros(...) non serve: il valore di c viene sostituito completamente.

Funzioni, condizioni e stampa

Separare il calcolo riutilizzabile dallo script che prepara gli input e mostra gli output.

1. Funzione in un file separato

```
function c = combinav(a,b)
    % Ipotesi: vettori colonna della stessa lunghezza
    n = length(a);
    c = zeros(n,1);
    for i = 1:n
        c(i) = 2*a(i) - b(i);
    end
end
```

Salva come combinav.m. La prima riga dichiara output c, nome combinav e input a,b. Le variabili della funzione sono locali: i nomi esterni possono essere diversi.

```
% File separato: test_combinav.m
clear; clc;
u = rand(8,1);
v = rand(8,1);
r = combinav(u,v);
disp(r);
```

Non richiamare una funzione che richiede input senza passarli. Per due output: function [a,b] = nome(n), poi [x,y] = nome(10).

2. Condizioni: if / elseif / else

```
if n < 0
    f = [];
elseif n == 0
    f = 1;
else
    f = prod(1:n);
end
```

Viene eseguito solo il primo ramo la cui condizione e vera; else copre i casi restanti. = assegna, == confronta. Altri confronti: ~=, <, >, <=, >=. Il fattoriale del corso presume un input intero: i decimali non sono controllati dal codice della docente.

3. fprintf: testo e valori nella stessa stampa

Formato	Significato
%d	Formato intero decimale; qui usato per indici e piccoli risultati interi.
%.6f	Formato decimale fisso con sei cifre dopo il separatore.
%8.6f	Come sopra, con larghezza minima di otto caratteri.
%.16e	Notazione scientifica con sedici cifre dopo il separatore.
\n	A capo.

```
fprintf('Indice %d, valore %.16e\n', i, d);
```

La formattazione cambia solo la visualizzazione, non la precisione del numero conservato. Non stampare un risultato vuoto come se fosse un valore scalare.

Accumulatori e prodotto matriciale

Schema comune: inizializza, aggiorna nel ciclo, restituisci il risultato.

1. Somma e prodotto: inizializzazioni diverse

```
% Prodotto: n! per n intero non negativo
f = 1;
for k = 1:n
    f = f*k;
end

% Somma armonica: 1 + 1/2 + ... + 1/n
s = 0;
for k = n:-1:1
    s = s + 1/k;
end
```

Un prodotto parte da 1; una somma parte da 0. Con n = 0, il ciclo 1:0 e vuoto e f rimane 1. Non azzerare l'accumulatore dentro il ciclo.

2. Ordine delle somme

Nel test con n = 10000: avanti = 9.7876060360443482; indietro = 9.7876060360443855. La differenza osservata e circa -3.73e-14. Gli arrotondamenti intermedi dipendono dall'ordine: l'addizione floating point non e in generale associativa. La differenza tra i risultati non e il loro errore rispetto al valore esatto.

3. Prodotto riga-colonna

Se A ha dimensioni m per p e B ha dimensioni p per n, C = A*B ha dimensioni m per n. C(i,j) e la somma, per k da 1 a p, dei prodotti A(i,k)*B(k,j). A.*B e invece un prodotto elemento per elemento.

```
function C = prodotto_matrici(A,B)
    [na,ma] = size(A);
    [nb,mb] = size(B);
    if ma == nb
        C = zeros(na,mb);
        for i = 1:na
            for j = 1:mb
                for k = 1:ma
                    C(i,j) = C(i,j) + A(i,k)*B(k,j);
                end
            end
        end
    else
        disp('Matrici non compatibili');
        C = [];
    end
end
```

i sceglie la riga di C; j la colonna; k completa la sommatoria per quell'elemento. Verifica con A = [1 2 3;4 5 6] e B = [7 8;9 10;11 12]: risultato [58 64;139 154].

Cancellazione e forma di Horner

Formule equivalenti in matematica possono comportarsi diversamente in precisione finita.

1. Il polinomio studiato

```
p1 = @(x) (x-1).^6;
p2 = @(x) x.^6 - 6*x.^5 + 15*x.^4 ...
          - 20*x.^3 + 15*x.^2 - 6*x + 1;
p3 = @(x) 1 + x.*(-6 + x.*(15 + x.*(-20 ...
          + x.*(15 + x.*(-6 + x)))));
x = linspace(0.996,1.004,10000);
plot(x,p1(x),x,p2(x),x,p3(x));
legend('fattorizzata','sviluppata','Horner');
```

Le tre espressioni rappresentano (x-1)^6. Vicino a 1 il risultato e minuscolo: per x = 1.001 e circa 1e-18. Nella forma sviluppata, termini molto piu grandi devono quasi cancellarsi; i loro errori possono dominare il risultato. Nel test le curve sviluppata e di Horner risultano frastagliate, quella fattorizzata regolare.

La forma fattorizzata e preferibile in questo esempio e intervallo; non e una regola universale. Una curva liscia, da sola, non dimostra accuratezza. La cancellazione e problematica quando amplifica, in termini relativi, errori gia presenti nelle quantita sottratte.

2. Mini parentesi: che cos'e la forma di Horner?

Non e una nuova funzione: e un modo annidato di scrivere e valutare un polinomio. Esempio:

```
p(x) = 2*x^3 - 3*x^2 + 4*x - 5
     = ((2*x - 3)*x + 4)*x - 5
```

Parti dal coefficiente della potenza piu alta. A ogni passo moltiplichi per x e aggiungi il coefficiente successivo. Non calcoli separatamente x^2 e x^3. Un polinomio di grado n richiede n moltiplicazioni e n addizioni nello schema standard.

```
% Coefficienti in ordine di potenze decrescenti
a = [2 -3 4 -5];
x = linspace(-2,2,100);
y = a(1)*ones(size(x));
for k = 2:length(a)
    y = y.*x + a(k);
end
% Comando MATLAB utile per lo stesso polinomio:
y_matlab = polyval(a,x);
```

Se manca una potenza, inserisci il coefficiente zero: x^3 + 2 corrisponde a [1 0 0 2]. polyval valuta il polinomio a partire dai coefficienti; non richiede il Symbolic Math Toolbox.

Horner migliora l'efficienza, ma non elimina tutte le cancellazioni. Nel nostro p3, le somme interne restano delicate vicino a x = 1. Meno operazioni non significa automaticamente maggiore accuratezza in ogni caso.

Determinante, underflow e logaritmi

L'ultimo esercizio mostra i limiti dell'intervallo di valori rappresentabile, non solo il numero di cifre disponibili.

1. Prodotto degli autovalori

In aritmetica esatta, il determinante di una matrice quadrata e il prodotto dei suoi autovalori contati con molteplicita. eig restituisce autovalori approssimati; det usa una fattorizzazione LU, non il prodotto degli autovalori calcolati da eig.

```
load matriceB.mat
v = eig(B);
d = 1;
for i = 1:length(v)
    d = d*v(i);
    fprintf('Primi %d autovalori: %.16e\n',i,d);
end
fprintf('det(B): %.16e\n',det(B));
L = sum(log10(abs(v)));
fprintf('Log10 del modulo del prodotto: %.16f\n',L);
```

2. Cosa abbiamo osservato

Nel tuo output: dopo 86 fattori d era circa 2.1047e-320; al fattore 87 e diventato zero. Anche det(B) ha restituito zero. La somma dei logaritmi era -400.0000000000002274: il modulo del prodotto degli autovalori calcolati e quindi circa 10^(-400). Non e una prova del valore esatto del determinante.

Soglia double	Valore / significato
realmin	Circa 2.2251e-308: minimo positivo normalizzato.
eps(0)	Circa 4.9407e-324: minimo positivo rappresentabile, includendo i subnormali.
Subnormali	Valori positivi sotto realmin: precisione relativa progressivamente ridotta.
Underflow a zero	Un valore sufficientemente piccolo viene arrotondato a zero.
Overflow	Un risultato supera il massimo valore finito rappresentabile.

3. Perche usare i logaritmi?

log10 del prodotto dei moduli = somma dei log10 dei moduli. Conservando L, eviti il prodotto minuscolo; eseguire 10^L torna invece a zero. Il modulo elimina segno e fase: per un determinante negativo o complesso queste informazioni vanno gestite separatamente. Un autovalore numerico zero produce log10(0) = -Inf.

Un determinante numerico nullo non dimostra singolarita. Per valutare la vicinanza alla singolarita usa strumenti come cond o rcond; anche rank e una valutazione numerica con una tolleranza. Non dedurre il rango dal solo det(B).

4. Fonti e ambito della scheda

Riepilogo dei sette script condivisi e dei risultati ottenuti durante la settimana 2. Le funzioni didattiche presumono gli input dichiarati; non sono implementazioni generali con validazione completa.

Riferimenti MathWorks: Create Functions in Files; zeros; Preallocation; mtimes; fprintf; A Brief History of polyval; Floating Point Numbers; Floating Point Denormals; det. Horner e polyval sono inclusi come breve approfondimento della forma annidata studiata.